

Question 1:What is Python? Explain any five important features of Python and mention why Python is considered beginner-friendly

Answer :- Python is a high-level, interpreted, dynamically typed, and general-purpose programming language created by Guido van Rossum and released in 1991.

It is designed with a strong focus on code readability, simplicity, and efficiency, enabling developers to write clear and logical code in fewer lines compared to languages like C++ or Java.

2. Five Important Features of Python

1. **Simple and Easy to Read:** Python uses plain English-like syntax and relies on clean indentation instead of complex curly braces `{ }` or semicolons `;`.
 2. **Interpreted Language:** Python code is executed line-by-line, making debugging fast and easy since errors are reported immediately at runtime.
 3. **Dynamically Typed:** You don't need to declare variable types (e.g., `int`, `float`) explicitly. Python automatically detects data types at runtime.
 4. **Extensive Standard Library:** It comes with a huge built-in library ("batteries included") for web development, data analysis, math operations, and file handling without needing extra code.
 5. **Cross-Platform / Portable:** Python code written on Windows can run smoothly on macOS, Linux, or Unix without any modifications.
-

3. Why Python is Considered Beginner-Friendly

- **Clean Syntax:** Reads almost like plain English, reducing clutter and syntax confusion for beginners.
- **Focus on Logic:** Allows new learners to focus on problem-solving rather than strict syntax or memory management.
- **No Complex Setup/Boilerplate:** A single line `print("Hello, World!")` runs immediately, unlike C++ or Java which require multiple lines of boilerplate code.
- **Automatic Memory Management:** Internal garbage collection handles memory allocation automatically.
- **Large Community Support:** Beginners can easily find solutions, documentation, and tutorials for any issue online.

Question 2: What is a variable in Python? Explain dynamic typing and describe the following Python data types with one example each: int, float, str, bool, list, tuple, dict, set

Answer :- A variable in Python is a named location in memory used to store data or values. Unlike languages like C or Java where variables are fixed containers, in Python, variables act as references or labels pointing to objects in memory.

- Syntax: `variable_name = value`
- Example:
- Python

`age = 20` # 'age' is a variable referencing the integer value 20

- -
-

2. Dynamic Typing in Python

Python is a dynamically typed language, meaning:

1. You do not need to explicitly specify the data type of a variable when declaring it.
 2. Python automatically determines the data type of a variable at runtime based on the assigned value.
 3. A variable can hold different data types at different points during program execution.
- Example:
 - Python

`x = 10` # 'x' is automatically assigned type 'int'

`x = "Hello"` # 'x' dynamically changes to type 'str' without throwing an error

- -
-

3. Python Data Types

1. `int` (Integer)

Represents whole numbers without any decimal point (positive, negative, or zero).

- Example:
- Python

```
count = 50
```

-
-

2. **float** (Floating-Point)

Represents numbers containing fractional parts or decimal points.

- Example:
- Python

```
price = 99.99
```

-
-

3. **str** (String)

Represents a sequence of characters enclosed within single (' '), double (" "), or triple quotes.

- Example:
- Python

```
language = "Python"
```

-
-

4. **bool** (Boolean)

Represents logic and truth values. It can only take one of two values: **True** or **False**.

- Example:
- Python

```
is_active = True
```

-
-

5. **list**

An ordered and mutable (changeable) collection of elements enclosed in square brackets `[ ]`. It allows duplicate items.

- Example:
- Python

```
fruits = ["apple", "banana", "mango"]
```

-
-

6. **tuple**

An ordered and immutable (unchangeable) collection of elements enclosed in round brackets `( )`. It allows duplicate items.

- Example:
- Python

```
coordinates = (10, 20)
```

-
-

7. **dict** (Dictionary)

A collection of key-value pairs enclosed in curly braces `{ }`. Keys must be unique, and values are accessed using their corresponding keys.

- Example:
- Python

```
student = {"name": "Rahul", "marks": 85}
```

-
-

8. **set**

An unordered collection of unique elements enclosed in curly braces `{ }`. It automatically removes duplicate values.

- Example:

- Python
- `unique_ids = {101, 102, 103}`

Question 3: Explain the following types of operators in Python with suitable examples for Arithmetic operators, Comparison operators, Logical operators, Assignment operators, Membership operators, Identity operators. Also explain the difference between implicit and explicit type conversion.

Answer :- **Part 1: Types of Operators in Python**

An **operator** in Python is a special symbol used to perform operations on variables and values (called operands).

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, etc.

Operator	Name	Description	Example (a = 10, b = 3)	Result
+	Addition	Adds two operands	<code>a + b</code>	13
-	Subtraction	Subtracts right operand from left	<code>a - b</code>	7
*	Multiplication	Multiplies two operands	<code>a * b</code>	30
/	Division	Divides left by right (always returns float)	<code>a / b</code>	3.333...

%	Modulus	Returns remainder of division	a % b	1
**	Exponentiation	Calculates power (\$a^b\$)	a ** b	1000
//	Floor Division	Divides and rounds down to nearest whole integer	a // b	3

2. Comparison (Relational) Operators

Comparison operators compare two values and return a Boolean value (True or False).

Operator	Name	Example (x = 5, y = 10)	Result
==	Equal to	x == y	False
!=	Not equal to	x != y	True
>	Greater than	x > y	False
<	Less than	x < y	True
>=	Greater than or equal to	x >= y	False
<=	Less than or equal to	x <= y	True

3. Logical Operators

Logical operators are used to combine conditional statements.

- **and**: Returns **True** if **both** statements are true.
- **or**: Returns **True** if **at least one** statement is true.
- **not**: Reverses the logical state (returns **False** if result is true).
- **Example Code:**
- Python

```
x = 10
# Logical AND
print(x > 5 and x < 20) # Output: True

# Logical OR
print(x > 15 or x == 10) # Output: True

# Logical NOT
print(not (x == 10)) # Output: False
•
•
```

4. Assignment Operators

Assignment operators are used to assign values to variables. They include simple assignment as well as compound operators that perform an operation and assignment simultaneously.

Operator	Example	Equivalent To	If `x = 10`, Result
:	<code>x = 5</code>	<code>x = 5</code>	5
+=	<code>x += 3</code>	<code>x = x + 3</code>	13
-=	<code>x -= 2</code>	<code>x = x - 2</code>	8
*=	<code>x *= 4</code>	<code>x = x * 4</code>	40
/=	<code>x /= 2</code>	<code>x = x / 2</code>	5.0
//=	<code>x //= 3</code>	<code>x = x // 3</code>	3

5. Membership Operators

Membership operators are used to test whether a value or sequence is present in an object (such as strings, lists, tuples, or sets).

*** * * in * * ***: Returns **True** if the specified value is present in the sequence.

* **`not in`**: Returns `True` if the specified value is ****not**** present in the sequence.

* **Example Code:**

```
```python
numbers = [10, 20, 30, 40]

print(20 in numbers) # Output: True
print(50 not in numbers) # Output: True
```

---

## 6. Identity Operators

Identity operators compare the memory locations (addresses) of two objects, not just their values.

- **is**: Returns **True** if both variables point to the **exact same object in memory**.
- **is not**: Returns **True** if both variables point to **different objects in memory**.
- **Example Code:**
- Python

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
```

```
print(a is c) # Output: True (c points to same memory location as a)
print(a is b) # Output: False (b is a new list with same values)
print(a == b) # Output: True (values are equal)
```

- 
- 

---

### ## Part 2: Implicit vs. Explicit Type Conversion

Type conversion (or Typecasting) is the process of converting one data type into another.

#### ### 1. Implicit Type Conversion (Automatic)

\* \*\*Definition:\*\* Performed automatically by the Python interpreter without any user intervention to prevent data loss.

\* \*\*Working:\*\* Python converts lower data types (like integer) to higher data types (like float) automatically during operations.

\* \*\*Example:\*\*

```
```python
```

```

num_int = 10 # int
num_float = 2.5 # float

# Python implicitly converts num_int to float before addition
result = num_int + num_float

print(result) # Output: 12.5
print(type(result)) # Output: <class 'float'>

```

2. Explicit Type Conversion (Typecasting)

- **Definition:** Manually performed by the programmer using built-in functions like `int()`, `float()`, `str()`, etc.
- **Working:** Used when a specific data type is required or when automatic conversion is not supported (e.g., converting a string of digits into an integer).
- **Example:**
- Python

```

num_str = "100" # string
num_int = 20 # int

# Explicit conversion of string to int using int() function
num_str_converted = int(num_str)

result = num_str_converted + num_int

print(result) # Output: 120
print(type(result)) # Output: <class 'int'>

```

Summary Comparison Table

Feature	Implicit Type Conversion	Explicit Type Conversion
---	---	---
Performed By	Python Interpreter automatically	Programmer manually using functions
Data Loss Risk	No risk (promotes to higher data type)	Risk of data loss (e.g., <code>int(12.9)</code> loses decimal <code>.9</code>)
Requirement	No special functions required	Uses built-in functions like <code>int()</code> , <code>float()</code> , <code>str()</code>
Example	<code>5 + 2.0</code> becomes <code>7.0</code>	<code>int("5") + 2</code> becomes <code>7</code>

Question 4:What is a string in Python? Explain string indexing, slicing, concatenation, repetition and formatting with suitable examples

Answer :- A string in Python is a sequence of characters enclosed within single quotes ('...'), double quotes ("..."), or triple quotes ('''...''' / """...""").

Strings are immutable, meaning once a string is created, its elements cannot be modified or changed in place.

- Example:
- Python

```
text1 = 'Hello'
text2 = "Python"
text3 = """This is a multi-line string."""
```

- -
-

2. String Concepts and Operations

A. String Indexing

Indexing allows you to access individual characters in a string using their position (index). Python uses zero-based indexing.

- Positive Indexing: Starts from 0 from the left end.
- Negative Indexing: Starts from -1 from the right end (backward indexing).
- Example Code:
- Python

```
word = "PYTHON"
```

```
print(word[0]) # Output: 'P' (First character)
print(word[2]) # Output: 'T'
print(word[-1]) # Output: 'N' (Last character)
print(word[-2]) # Output: 'O'
```

-
-

B. String Slicing

Slicing is used to extract a range or sub-string from a string using the syntax `string[start:stop:step]`.

- **start:** Starting index (inclusive)
- **stop:** Ending index (exclusive)
- **step:** Step size/increment (default is 1)
- Example Code:
- Python

```
text = "Hello World"
```

```
print(text[0:5]) # Output: 'Hello' (From index 0 to 4)
print(text[6:]) # Output: 'World' (From index 6 to end)
print(text[:5]) # Output: 'Hello' (From start to index 4)
print(text[::2]) # Output: 'HloWrld' (Every 2nd character)
print(text[::-1]) # Output: 'dlroW olleH' (Reverses the string)
```

-
-

C. String Concatenation

String concatenation is the process of joining two or more strings together using the addition operator (+).

- Example Code:
- Python

```
first_name = "Guido"
last_name = "van Rossum"
```

```
full_name = first_name + " " + last_name
print(full_name) # Output: Guido van Rossum
```

- -
-

D. String Repetition

String repetition is used to repeat a string a specified number of times using the multiplication operator (*).

- Example Code:
- Python

```
word = "Ha"  
result = word * 3
```

```
print(result) # Output: HaHaHa
```

- -
-

E. String Formatting

String formatting allows you to insert dynamic variables and values inside a string.

1. f-strings (Formatted String Literals - Modern & Preferred): Uses **f** before the quotes and variable names inside **{ }** placeholders.
2. Python

```
name = "Rahul"  
age = 20  
print(f"My name is {name} and I am {age} years old.")  
# Output: My name is Rahul and I am 20 years old.
```

- 3.
- 4.
5. **.format()** Method: Uses **{ }** as placeholders filled by values passed into **.format()**.
6. Python

```
name = "Rahul"  
marks = 85.5  
print("Student {} scored {} marks.".format(name, marks))  
# Output: Student Rahul scored 85.5 marks.
```

7. **% Operator (Old Style)**: Uses placeholders like **%s** (string) and **%d** (integer).

8. Python

```
item = "Apple"  
price = 50  
print("The price of %s is %d rupees." % (item, price))
```

9. # Output: The price of Apple is 50 rupees.

10. % Operator (Old Style): Uses placeholders like %s (string) and %d (integer).

11. Python

```
item = "Apple"  
price = 50  
print("The price of %s is %d rupees." % (item, price))
```

12. # Output: The price of Apple is 50 rupees.

Question 5: What is a list in Python? Explain why lists are called mutable. Describe any five commonly used list methods with examples.

Answer :- A **list** in Python is a built-in, **ordered**, and **mutable** collection of elements enclosed in square brackets ([]). Lists can store items of mixed data types (integers, strings, floats, booleans, or even other lists) and allow duplicate values.

- **Syntax:** `list_name = [item1, item2, item3]`
- **Example:**
- Python

```
my_list = [10, "Python", 3.14, True]
```

-
-

2. Why Lists are Called Mutable

In Python, **mutability** means that an object's state or content can be changed, modified, added, or removed **in place** after it has been created, without changing its original memory address (`id`).

Lists are called **mutable** because you can alter their elements, append new items, or delete existing items directly without creating a completely new list in memory.

- **Example demonstrating mutability:**
- Python

```
numbers = [10, 20, 30]
print(numbers) # Output: [10, 20, 30]
```

```
# Modifying an element at index 1
numbers[1] = 99
print(numbers) # Output: [10, 99, 30] (Modified directly in place)
```

- -
-

3. Five Commonly Used List Methods

1. `append()`

- **Description:** Adds a single element to the very **end** of the list.
- **Example:**
- Python

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)
# Output: ['apple', 'banana', 'cherry']
```

- -
-

2. `insert()`

- **Description:** Inserts an element at a **specific index/position** in the list (`list.insert(index, element)`).

- **Example:**
- Python

```
numbers = [1, 3, 4]
numbers.insert(1, 2) # Inserts '2' at index 1
print(numbers)
# Output: [1, 2, 3, 4]
```

- -
-

3. **remove()**

- **Description:** Removes the **first occurrence** of a specified value from the list.
- **Example:**
- Python

```
colors = ["red", "blue", "green", "blue"]
colors.remove("blue") # Removes the first "blue"
print(colors)
# Output: ['red', 'green', 'blue']
```

- -
-

4. **pop()**

- **Description:** Removes and returns the item at a given index (if no index is specified, it removes and returns the **last item**).
- **Example:**
- Python

```
items = [10, 20, 30, 40]
removed_item = items.pop(2) # Removes item at index 2
print(removed_item) # Output: 30
print(items) # Output: [10, 20, 40]
```

- -
-

5. `sort()`

- **Description:** Sorts the elements of the list in **ascending order** by default (modifies the list in place).
- **Example:**
- Python

```
scores = [45, 12, 89, 32]
scores.sort()
print(scores)
```

- # Output: [12, 32, 45, 89]
-

Question 6: Write a Python program that accepts two numbers from the user and performs: • Addition • Subtraction • Multiplication • Division • Remainder Display all results clearly

Answer :- Python

Program to perform basic arithmetic operations on two numbers

Taking two numbers as input from the user

```
num1 = float(input("Enter the first number: "))
```

```
num2 = float(input("Enter the second number: "))
```

Performing arithmetic operations

```
addition = num1 + num2
```

```
subtraction = num1 - num2
```

```
multiplication = num1 * num2
```

Displaying results

```
print("\n--- RESULTS ---")
```

```
print(f"Addition ({num1} + {num2})    = {addition}")
```

```
print(f"Subtraction ({num1} - {num2}) = {subtraction}")
```

```
print(f"Multiplication ({num1} * {num2}) = {multiplication}")
```

Handling division by zero safely

```
if num2 != 0:
```

```
    division = num1 / num2
```

```
    remainder = num1 % num2
```

```
    print(f"Division ({num1} / {num2})    = {division}")
```

```
    print(f"Remainder ({num1} % {num2})    = {remainder}")
```

```
else:
```

```
print("Division (/)      = Error! Cannot divide by zero.")
print("Remainder (%)     = Error! Cannot perform modulus by zero.")
```

Output Example

Plaintext

Enter the first number: 15

Enter the second number: 4

--- RESULTS ---

Addition (15.0 + 4.0) = 19.0

Subtraction (15.0 - 4.0) = 11.0

Multiplication (15.0 * 4.0) = 60.0

Division (15.0 / 4.0) = 3.75

Remainder (15.0 % 4.0) = 3.0

Question 7: Write a Python program that accepts a sentence from the user and displays:

1. The sentence in uppercase.
2. The sentence in lowercase.
3. The total number of characters.
4. The first five characters.
5. The reversed sentence.
6. The number of words in the sentence..

Answer :- Python Code

Python

Program to perform various string operations on a user-input sentence

Accepting a sentence from the user

sentence = input("Enter a sentence: ")

```
# 1. Converting the sentence to uppercase
uppercase_sentence = sentence.upper()

# 2. Converting the sentence to lowercase
lowercase_sentence = sentence.lower()

# 3. Calculating the total number of characters (including spaces)
total_characters = len(sentence)

# 4. Extracting the first five characters using slicing
first_five = sentence[:5]

# 5. Reversing the sentence using step slicing [::-1]
reversed_sentence = sentence[::-1]

# 6. Counting the total number of words (split by whitespace)
word_count = len(sentence.split())

# Displaying all results clearly
print("\n--- RESULTS ---")
print(f"1. Uppercase Sentence      : {uppercase_sentence}")
print(f"2. Lowercase Sentence       : {lowercase_sentence}")
print(f"3. Total Number of Characters: {total_characters}")
print(f"4. First Five Characters    : {first_five}")
print(f"5. Reversed Sentence        : {reversed_sentence}")
print(f"6. Number of Words          : {word_count}")
```

Output Example

Plaintext

Enter a sentence: Python Programming is fun

--- RESULTS ---

```
1. Uppercase Sentence      : PYTHON PROGRAMMING IS FUN
2. Lowercase Sentence       : python programming is fun
3. Total Number of Characters: 25
4. First Five Characters    : Pytho
5. Reversed Sentence        : nuf si gnimmargorP nohtyP
6. Number of Words          : 4
```

Question 8: Create the following list: marks = [72, 85, 60, 91, 78]
Write a Python program to:

1. Add 88 to the list.
2. Remove 60.
3. Sort the list.
4. Display the highest mark
- . 5. Display the lowest mark.
6. Calculate the average mark.

Answer :- Python

Initializing the marks list

marks = [72, 85, 60, 91, 78]

1. Add 88 to the list using append()

marks.append(88)

2. Remove 60 from the list using remove()

marks.remove(60)

3. Sort the list in ascending order

marks.sort()

4. Display the highest mark using max()

highest_mark = max(marks)

5. Display the lowest mark using min()

lowest_mark = min(marks)

6. Calculate the average mark (sum of elements / total count)

average_mark = sum(marks) / len(marks)

Displaying all results

print("Updated and Sorted Marks List :", marks)

print("1. Highest Mark : ", highest_mark)

```
print("2. Lowest Mark          :", lowest_mark)
print(f"3. Average Mark       : {average_mark:.2f}")
```

Output Example

Plaintext

Updated and Sorted Marks List : [72, 78, 85, 88, 91]

```
1. Highest Mark      : 91
2. Lowest Mark       : 72
3. Average Mark      : 82.80
```

Question 9: Consider the following string:

"Python Java C C++ Python Java Python" Write a Python program to:

- 1. Convert the string into a list of programming languages.**
- 2. Display the list.**
- 3. Count how many times "Python" appears.**
- 4. Remove duplicate language names.**
- 5. Join the unique language names using " | ".**

Answer :- Python Code

Python

Initializing the given string

text = "Python Java C C++ Python Java Python"

1. Convert the string into a list of programming languages using split()

lang_list = text.split()

2. Display the list

print("1. List of languages :", lang_list)

3. Count how many times "Python" appears using count()

```
python_count = lang_list.count("Python")
print("2. Count of 'Python'      :", python_count)
```

```
# 4. Remove duplicate language names (dict.fromkeys preserves order)
unique_langs = list(dict.fromkeys(lang_list))
print("3. List without duplicates  :", unique_langs)
```

```
# 5. Join the unique language names using " | "
result_string = " | ".join(unique_langs)
print("4. Joined unique languages   :", result_string)
```

Output Example

Plaintext

```
1. List of languages      : ['Python', 'Java', 'C', 'C++', 'Python', 'Java', 'Python']
2. Count of 'Python'      : 3
3. List without duplicates : ['Python', 'Java', 'C', 'C++']
4. Joined unique languages : Python | Java | C | C++
```

Question 10: Write a Python program to store the following student information:

- Student name
- Course name
- Marks in three subjects

The program should:

1. Store the three marks in a list.
2. Calculate the total marks.
3. Calculate the average.
4. Display the highest mark.
5. Display all student details using an f-string.

Answer :- **Python Code**

Python

Program to store student information and calculate results

1. Storing student details and taking three marks in a list

student_name = input("Enter Student Name: ")

course_name = input("Enter Course Name: ")

Input marks for three subjects

mark1 = float(input("Enter marks for Subject 1: "))

mark2 = float(input("Enter marks for Subject 2: "))

mark3 = float(input("Enter marks for Subject 3: "))

Store the three marks in a list

marks_list = [mark1, mark2, mark3]

2. Calculate the total marks

total_marks = sum(marks_list)

```

# 3. Calculate the average marks
average_marks = total_marks / len(marks_list)

# 4. Find the highest mark
highest_mark = max(marks_list)

# 5. Displaying all student details using an f-string
print("\n" + "="*35)
print("      STUDENT REPORT CARD      ")
print("="*35)
print(f"Student Name : {student_name}")
print(f"Course Name  : {course_name}")
print(f"Marks List   : {marks_list}")
print(f"Total Marks   : {total_marks:.2f}")
print(f"Average Marks : {average_marks:.2f}")
print(f"Highest Mark  : {highest_mark:.2f}")
print("="*35)

```

Output Example

Plaintext

```

Enter Student Name: Rahul Sharma
Enter Course Name: Computer Science
Enter marks for Subject 1: 85
Enter marks for Subject 2: 92
Enter marks for Subject 3: 78

```

```

=====
      STUDENT REPORT CARD
=====
Student Name : Rahul Sharma
Course Name  : Computer Science
Marks List   : [85.0, 92.0, 78.0]
Total Marks   : 255.00
Average Marks : 85.00
Highest Mark  : 92.00
=====

```

